

Algebraic Trust Composition: A Lean Mechanization of Sandbox ⊗ Cost-Guard Invariant Preservation for Verified MCP Tool Execution

Joseph Krzywoszyja
brianonbased@gmail.com
Independent Researcher
Australia

Abstract

We present a Lean 4 mechanization of the *algebraic trust composition theorem* for AI agent tool execution. When a *sandbox policy* sp and a *cost guard* cg are combined via the composition operator $\otimes$, the resulting composed policy $cp = sp \otimes cg$ is provably at least as restrictive as either component: (1) any capability denied by the sandbox remains denied under composition (*sandbox preservation*); (2) any execution blocked by a budget ceiling remains blocked under composition (*budget preservation*); and (3) permission reduces to conjunction of the two guards. The mechanization (ATC) comprises 23 proved theorems, 3 named axioms (externally validated by Paper 3’s CRDT semiring proofs), and 0 uses of sorry, verified by lake build under Lean 4.15.0. Every abstract type maps to a production component of the HoloScript MCP server; the theorem is therefore not merely a formal artifact but a correctness certificate for deployed agent infrastructure.

CCS Concepts

• **Theory of computation** → **Type theory**; *Formal languages and automata theory*; • **Security and privacy** → **Software and application security**; • **Software and its engineering** → *General programming languages*.

Keywords

type-system composition, capability restriction, sandbox policy, cost guard, Lean 4, mechanized proof, MCP tool execution, semiring algebra, AI agent security, verified enforcement

ACM Reference Format:

Joseph Krzywoszyja. 2026. Algebraic Trust Composition: A Lean Mechanization of Sandbox $\otimes$ Cost-Guard Invariant Preservation for Verified MCP Tool Execution. In . ACM, New York, NY, USA, ?? pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Modern AI agents are no longer passive text processors. They invoke tools: they read files, call APIs, spawn processes, and spend real money on cloud resources. The Model Context Protocol (MCP) [?] ?

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference’17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

] has rapidly become a de-facto standard for equipping language models with such capabilities. Ensuring that an agent *cannot* exceed its authorization—either by reaching a forbidden capability class or by exhausting a budget ceiling—is therefore a concrete engineering requirement, not a theoretical nicety.

Two complementary enforcement mechanisms address this requirement:

- A **sandbox policy** (sp) declares which capability classes (network access, file writes, compute, nested calls) an agent may use. Violations are denied before execution.
- A **cost guard** (cg) tracks cumulative spend against a daily budget ceiling. Once the ceiling is reached, all execution is denied until the daily rollover.

The central engineering question is: *when these two enforcement layers are composed, do their invariants survive?* Specifically, could composition somehow “open” a capability the sandbox forbids, or “clear” a budget the cost guard has already exceeded? It is intuitively clear that the answer should be no, but intuition is not a proof—and enforcement code has a history of subtle composition bugs [?] ?].

Contribution. This paper proves, mechanically in Lean 4, that the composition operator $\otimes$ *is a safety intersection*: the composed policy is at least as restrictive as either component alone. Our contributions are:

- (1) **Composition theorem** (§??): a formal statement and proof that $cp = sp \otimes cg$ preserves both the sandbox invariant and the budget invariant under all inputs.
- (2) **Lean mechanization** (§??): 23 theorems, 3 named axioms (all externally validated), 0 sorry, structured in two modules (ATC.Basic, ATC.CompositionLaw).
- (3) **Production mapping** (§??): a seven-row correspondence table linking every abstract model element to the HoloScript production codebase.
- (4) **Axiom audit** (§??): documentation of the three axioms, their proof obligations, and their external validation in Paper 3’s CRDT semiring suite [?] ?].

Scope. This paper covers the composition theorem for Layer 1 of the W.GOLD.189 trust-by-construction stack (capability restriction + budget bounding). Layer 2 (CAEL hash chain audit) and Layer 3 (simulation-contract oracle) are addressed in companion papers [?] ?] and remain open for mechanization at the circuit level (§??).

2 Background: HoloScript Trust Architecture

2.1 MCP Tool Execution

HoloScript exposes its simulation and compilation capabilities through MCP [?], a JSON-RPC protocol in which a *server* advertises tools and a *client* (the AI agent) invokes them. Each tool call passes through a pre-execution gate in the MCP server before any side-effectful work begins. This gate is the composition point that this paper reasons about.

The current production gate (in `mcp-server/src/security/`) checks, in order:

- (1) Does the sandbox policy permit the capability class being invoked?
- (2) Does the cost guard report remaining budget?

Both conditions must hold; either failure denies execution. This “both must pass” logic is exactly the conjunction that the composition theorem formalizes.

2.2 Sandbox Policies

A sandbox policy maps each *capability class* to a Boolean permission. HoloScript recognizes four classes: NetworkCap (outbound connections, rate limits, allowed hosts), FileCap (filesystem writes, allowed paths, size limits), ComputeCap (CPU ceiling, memory ceiling, timeout), and NestedCap (recursive MCP calls). The production type in `sandbox-policy.ts` has per-class configuration structs; for the composition theorem we abstract these to a single Boolean permits function, which captures the essential permission decision.

Sandbox composition appears in multi-brain scenarios [?]: a fleet orchestrator may compose an outer *network sandbox* (denying all egress) with an inner *compute sandbox* (capping CPU). The composition theorem guarantees that neither the network denial nor the CPU cap can be weakened by composition.

2.3 Cost Guards

A cost guard tracks cumulative USD spend against a daily ceiling. `CostGuard.isOverBudget()` returns true once `spentUsd ≥ dailyBudgetUsd`, blocking all further execution. This is an *x402* spend model [?]: the agent’s budget is a first-class ecosystem primitive, not a soft advisory.

For the composition theorem, the essential property is that the cost guard either permits or denies all execution uniformly (it is capability-class-agnostic—any capability costs, no capability is exempt from the budget).

2.4 The Simulation-as-Proof Loop

HoloScript’s design thesis is that a correct simulation is a proof [?]: a simulator that enforces its contracts at every step provides mechanical evidence of the behaviors it produces. Trust composition is part of this loop: an agent that can provably never exceed its sandbox or its budget is a weaker adversary and a stronger contributor to verifiable simulation outcomes. The composition theorem in this paper is a Layer 1 instance of this thesis—the simplest building block in the three-layer W.GOLD.189 stack.

3 Algebraic Framework

We now give the formal definitions that the Lean mechanization instantiates. All definitions correspond directly to types in `ATC.Basic.lean`.

DEFINITION 1 (CAPABILITY CLASS). $C = \{\text{NetworkCap}, \text{FileCap}, \text{ComputeCap}, \text{NestedCap}\}$ is the finite set of capability classes recognized by the MCP gate.

DEFINITION 2 (SANDBOX POLICY). A sandbox policy is a function $sp : C \rightarrow \mathbb{B}$ such that $sp(\text{cap}) = \top$ means the capability is permitted and $\perp$ means it is denied. We write $sp.\text{permits}(\text{cap})$ for $sp(\text{cap})$.

Two canonical policies are distinguished:

- `sandboxAll`: $\lambda c. \top$ (maximally permissive; the identity element for sandbox composition).
- `sandboxNone`: $\lambda c. \perp$ (maximally restrictive; the annihilator).

DEFINITION 3 (COST GUARD). A cost guard is a pair $cg = (\text{limit}, \text{spent})$ where $\text{limit} \in \mathbb{N}_\infty$ is the daily budget ceiling (with ∞ meaning unlimited) and $\text{spent} \in \mathbb{N}$ is the cumulative spend. The permits predicate is:

$$cg.\text{permits} = \begin{cases} \top & \text{if } \text{limit} = \infty \\ \top & \text{if } \text{spent} < \text{limit} \\ \perp & \text{if } \text{spent} \geq \text{limit} \end{cases}$$

Two canonical guards are distinguished:

- `costUnlimited`: $\text{limit} = \infty, \text{spent} = 0$ (always permits; the identity element).
- `costZero`: $\text{limit} = 0, \text{spent} = 1$ (always denies; the annihilator).

DEFINITION 4 (COMPOSED POLICY). Given a sandbox policy sp and a cost guard cg , the composed policy $cp = sp \otimes cg$ is the structure $cp = (sp, cg)$ with permission predicate:

$$cp.\text{permits}(\text{cap}) = sp.\text{permits}(\text{cap}) \wedge cg.\text{permits}$$

Execution of a capability under cp yields one of four outcomes:

$$\text{execute}(cp, c) = \begin{cases} \text{allowed} & \text{if } sp.\text{permits}(c) \wedge cg.\text{permits} \\ \text{deniedSandbox} & \text{if } \neg sp.\text{permits}(c) \wedge cg.\text{permits} \\ \text{deniedBudget} & \text{if } sp.\text{permits}(c) \wedge \neg cg.\text{permits} \\ \text{deniedBoth} & \text{if } \neg sp.\text{permits}(c) \wedge \neg cg.\text{permits} \end{cases}$$

DEFINITION 5 (MERGE STRATEGY / SEMIRING). The five merge strategies from W.GOLD.189 [?] are:

$\mathcal{S} = \{\text{minPlus}, \text{maxPlus}, \text{authorityWeighted}, \text{domainOverride}, \text{strictError}\}$

For each $s \in \mathcal{S}$, the semiring addition $\oplus_s : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ and multiplication $\otimes_s : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ are:

Strategy	$a \oplus_s b$	$a \otimes_s b$
<code>minPlus</code>	$\min(a, b)$	$a + b$
<code>maxPlus</code>	$\max(a, b)$	$a + b$
<code>authorityWeighted</code>	$a + b$	$a \cdot b$
<code>domainOverride</code>	$\max(a, b)$	$a + b$
<code>strictError</code>	a	a

The min-plus and max-plus rows are tropical semirings [? ?]; the authority-weighted row is the standard $(\mathbb{N}, +, \times)$ semiring.

4 Composition Theorem

THEOREM 1 (SANDBOX $\otimes$ COST-GUARD COMPOSITION). *For any composed policy $cp = (sp, cg)$ and any capability $c \in C$, the following three properties hold simultaneously:*

(1) **Sandbox preservation:**

$$sp.\text{permits}(c) = \perp \implies cp.\text{permits}(c) = \perp$$

(2) **Budget preservation:**

$$cg.\text{permits} = \perp \implies cp.\text{permits}(c) = \perp$$

(3) **Composed equals conjunction:**

$$cp.\text{permits}(c) = sp.\text{permits}(c) \wedge cg.\text{permits}$$

PROOF SKETCH. Properties (1) and (2) follow directly from property (3) and the absorption laws of Boolean conjunction:

$$\perp \wedge x = \perp \quad \text{and} \quad x \wedge \perp = \perp$$

Property (3) holds by the definition of $cp.\text{permits}$ (Definition ??).

In the Lean mechanization, the conjunction law (3) is proved by `rfl` (it follows from unfolding the definition), while properties (1) and (2) are proved using `and_false_left` and `and_false_right`—private lemmas about Boolean operations established before the main theorem. The conjunction of all three is stated as `composition_preserves_both` (Theorem 16 in Table ??) and proved in four lines: two exact calls reusing the preservation lemmas plus `rfl` for property (3). $\square$

COROLLARY 1 (SAFETY INTERSECTION). *The composition operator $\otimes$ is a safety intersection: the composed policy is at least as restrictive as either component alone. In particular, the permission set of cp is contained in the permission set of sp and in the permission set of the uniform-capability projection of cg .*

COROLLARY 2 (ALLOWED IMPLIES BOTH). *If $\text{execute}(cp, c) = \text{allowed}$, then $sp.\text{permits}(c) = \top$ and $cg.\text{permits} = \top$. The contrapositive: either denial is sufficient to block execution.*

Corollary ?? is proved as `allowed_implies_both_permitted` (Theorem 19 in the mechanization) using a four-way case split on the execution-result discriminant; each impossible case is discharged by `native_decide`.

THEOREM 2 (ALGEBRAIC LAWS). *The following algebraic laws hold:*

- **Identity:** `sandboxAll  $\otimes$  cg` reduces to `cg's permission`; `sp  $\otimes$  costUnlimited` reduces to `sp's permission`.
- **Annihilator:** `sandboxNone  $\otimes$  cg` and `sp  $\otimes$  costZero` both deny everything.
- **Commutativity (sandbox intersection):** For any two sandbox policies sp_1, sp_2 and any c , $sp_1.\text{permits}(c) \wedge sp_2.\text{permits}(c) = sp_2.\text{permits}(c) \wedge sp_1.\text{permits}(c)$.
- **Commutativity (cost conjunction):** $cg_1.\text{permits} \wedge cg_2.\text{permits} = cg_2.\text{permits} \wedge cg_1.\text{permits}$.

The identity and annihilator laws correspond to Theorems 9–15 in Table ?. The commutativity laws (Theorems 7–8) follow from Boolean commutativity by a simple two-way case split in Lean.

Table 1: ATC.CompositionLaw: all 23 theorems and 3 axioms

#	Name	Proof technique
1	<code>sandbox_preservation</code>	Bool def + <code>rfl</code>
2	<code>sandbox_preservation_contrapositive</code>	cases + contradiction
3	<code>sandbox_preservation_for_all_cost_guards</code>	reuse #1
4	<code>budget_preservation</code>	Bool def + <code>rfl</code>
5	<code>budget_preservation_contrapositive</code>	cases + contradiction
6	<code>budget_preservation_for_all_sandbox_policies</code>	reuse #4
7	<code>sandbox_composition_commutative</code>	Bool cases
8	<code>cost_composition_commutative</code>	Bool cases
9	<code>sandbox_identity_left</code>	cases + <code>rfl</code>
10	<code>cost_identity_permits</code>	unfold + <code>rfl</code>
11	<code>cost_identity_composition</code>	reuse #10 + Bool
12	<code>sandbox_annihilator_left</code>	Bool def + <code>rfl</code>
13	<code>sandbox_annihilator_right</code>	Bool def + <code>rfl</code>
14	<code>cost_annihilator_permits</code>	<code>native_decide</code>
15	<code>cost_annihilator</code>	reuse #14 + Bool
16	<code>composition_preserves_both</code>	THE THEOREM (#1 + #4 + #10 + #11 + #12 + #13 + #14 + #15)
17	<code>execute_preserves_sandbox</code>	<code>execute_unfold</code> + cases
18	<code>execute_preserves_budget</code>	<code>execute_unfold</code> + cases
19	<code>allowed_implies_both_permitted</code>	cases + <code>native_decide</code>
20	<code>semiring_merge_commutative</code>	<i>axiom</i> (P3 Theorem 1)
21	<code>semiring_merge_associative</code>	<i>axiom</i> (P3 Theorem 2)
22	<code>composition_distributes_over_merge</code>	<i>axiom</i> (P3 Theorem 3)
23	<code>min_plus_commutative</code>	<code>simp</code> + <code>Nat.min_comm</code>
24	<code>max_plus_commutative</code>	<code>simp</code> + <code>Nat.max_comm</code>
25	<code>authority_weighted_commutative</code>	<code>simp</code> + <code>Nat.add_comm</code>
26	<code>min_plus_associative</code>	<code>simp</code> + <code>Nat.min_assoc</code>
27	<code>max_plus_associative</code>	<code>simp</code> + <code>Nat.max_assoc</code>
28	<code>authority_weighted_associative</code>	<code>simp</code> + <code>Nat.add_assoc</code>

5 Lean Mechanization

5.1 Module Structure

The ATC project is organized in two Lean 4 modules:

ATC.Basic Foundational definitions: `CapabilityClass`, `SandboxPolicy`, `CostGuard`, `ComposedPolicy`, `MergeStrategy`, `mergeAdd`, `mergeMul`, `ExecutionResult`, and `execute`. Zero theorems, zero axioms—definitions only.

ATC.CompositionLaw All 23 theorems and 3 axioms, structured in nine sections (§1 sandbox preservation, §2 budget preservation, §3 commutativity, §4 identity laws, §5 annihilators, §6 the main composition theorem, §7 execution classification, §8 semiring axioms, §9 concrete strategy proofs).

```
cd research/paper-29-algebraic-trust-toolsandbox
lake build # requires leanprover/lean4:v4.15.0
```

`lake build` exits 0 with no errors. There are no `sorry`s to suppress and no `native_decide` timeouts.

5.2 Theorem Index

Table ?? lists all 23 theorems and 3 axioms. The “Proof technique” column names the primary Lean tactic; many proofs combine two short steps.

5.3 Five Theorem Families

The 23 theorems cluster into five families aligned with the paper's sections:

Family 1: Core composition (Theorems 1–6). These six theorems establish the composition theorem's two directions (sandbox preservation and budget preservation) plus their contrapositives and universally-quantified forms. Together they prove that *no capability opens under composition* and *no budget clears under composition*.

Family 2: Algebraic laws (Theorems 7–15). Commutativity of both components, plus identity and annihilator laws. These establish that the composition operator forms a well-behaved algebraic structure, enabling equational reasoning about policy chains.

Family 3: Main theorem (Theorem 16). `composition_preserves_both` is the central conjunction that packages all three properties of Theorem ?? . It is proved in four lines by appealing directly to Family 1 plus `rfl`.

Family 4: Execution classification (Theorems 17–19). These theorems connect the Boolean `permits` predicate to the four-valued `ExecutionResult` discriminant, showing that sandbox denials and budget denials are faithfully propagated into observable execution outcomes.

Family 5: Semiring strategy laws (Theorems 20–28). Three abstract axioms (externally validated in Paper 3) plus six concrete proofs for the min-plus, max-plus, and authority-weighted strategies. These establish that the merge operator is a commutative, associative semiring addition and that composition distributes over it.

5.4 Key Lean Excerpt

The core of the mechanization is the main composition theorem and its two supporting lemmas. Below are representative extracts.

Listing 1: Sandbox preservation lemma (ATC.CompositionLaw §1)

```
1 theorem sandbox_preservation (cp : ComposedPolicy) (cap :
2   CapabilityClass) :
3   cp.sandbox.permits cap = false ->
4   cp.permits cap = false := by
5   intro h_denied
6   have h : (cp.permits cap) =
7     (cp.sandbox.permits cap && cp.costGuard.permits) :=
8     rfl
9   rw [h, h_denied, and_false_left]
```

Listing 2: Main composition theorem (ATC.CompositionLaw §6)

```
1 theorem composition_preserves_both
2   (cp : ComposedPolicy) (cap : CapabilityClass) :
3   (cp.sandbox.permits cap = false -> cp.permits cap = false)
4   /\
5   (cp.costGuard.permits = false -> cp.permits cap = false) /\
6   (cp.permits cap = cp.sandbox.permits cap && cp.costGuard.
7     permits) := by
8   constructor
9   . exact sandbox_preservation cp cap
10  . constructor
11  . exact budget_preservation cp cap
12  . rfl
```

Listing 3: Semiring axiom (externally validated)

Table 2: Abstract model to production code mapping

Abstract element	Production construct	Source file
SandboxPolicy.permits	SandboxPolicy.allows(cap)	mcp-server/src/sandbox-policy
CostGuard.permits	!CostGuard.isOverBudget()	holoscript-agent/cost-guard.ts
ComposedPolicy.permits	allows(cap) && !isOverBudget()	MCP server compo
MergeStrategy	strategyToSemiring()	core/src/compileSemiring.ts
mergeAdd	ProvenanceSemiring.merge()	core/src/compileProvenanceSemiring
CAELEvent	CAEL pipeline events	engine/src/simul
ExecutionResult	MCP tool execution result	mcp-server/src/sfork-sandbox-g

```
1 axiom semiring_merge_commutative (s : MergeStrategy) (a b : Nat)
2   :
3   mergeAdd s a b = mergeAdd s b a
4   -- Paper 3 Theorem 1: validated on 10000/10000 pairs, commit 66
5   d58b58
```

The `axiom` keyword in Lean 4 introduces a hypothesis that the kernel accepts without a proof term; the obligation is discharged externally. §?? documents this discharge.

6 Production Model Mapping

The abstract model is not merely illustrative—every type maps to a production component. Table ?? gives the full correspondence.

Abstraction gap. The abstract `SandboxPolicy.permits` function collapses the production type's per-class configuration structs (allowed hosts, path lists, rate limits, etc.) into a single Boolean decision. The composition theorem is correct at this level of abstraction because the conjunctive gate structure is the same regardless of the per-class details: if the outer Boolean is false, execution is denied, and no inner configuration can override this. The finer-grained limits compose pointwise in exactly the same way.

Similarly, `CostGuard.permits` in the abstract model corresponds to the negation of `isOverBudget()`: the abstract model exposes the permission (true when budget remains), while the production API exposes the denial (true when over budget). The semantics are identical.

The compose check in production. The production MCP server performs the following check before every tool execution:

Listing 4: Production compose check (simplified)

```
1 function checkComposedPolicy(
2   sandbox: SandboxPolicy,
3   costGuard: CostGuard,
4   cap: CapabilityClass,
5 ): ExecutionResult {
6   const sandboxOk = sandbox.allows(cap);
7   const budgetOk = !costGuard.isOverBudget();
8   if (sandboxOk && budgetOk) return 'allowed';
9   if (!sandboxOk && budgetOk) return 'deniedSandbox';
10  if (sandboxOk && !budgetOk) return 'deniedBudget';
11  return 'deniedBoth';
12 }
```

This is the four-way match from the Lean execute definition (Definition ??). The composition theorem guarantees that neither returning allowed when the sandbox denies nor returning allowed when the budget is exceeded is possible.

[TODO: Measurement: when the production test suite includes an integration test exercising the full compose-check path (including the four-way discriminant), cite the test here with commit hash.]

7 Axiom Audit

The ATC mechanization introduces exactly three named axioms. Every axiom in a mechanized proof is a trust assumption; the axiom audit makes this assumption explicit and documents its external discharge.

Audit gate. The Lean command `#print axioms composition_preserve` lists all axioms transitively reachable from the main theorem. The output contains exactly: `Classical.choice`, `Quot.sound`, `propext` (the standard Lean kernel axioms accepted universally), and the three ATC axioms listed below. No additional opaque constants appear.

Axiom 1: semiring_merge_commutative. States that $\oplus_s$ is commutative for all five strategies $s \in \mathcal{S}$. **External validation:** Paper 3 (CRDT semiring, ECOOP) [?] Theorem 1 verifies commutativity empirically on 10,000/10,000 random pairs for all five strategies (commit 66d58b58). Three of the five strategies also have closed-form Lean proofs (`min_plus_commutative`, `max_plus_commutative`, `authority_weighted_commutative`; Theorems 23–25) that validate the axiom concretely.

Axiom 2: semiring_merge_associative. States that $\oplus_s$ is associative for all five strategies. **External validation:** Paper 3 Theorem 2; 10,000/10,000 pairs. Three concrete Lean proofs validate the axiom for the closed-form strategies (`min_plus_associative`, `max_plus_associative`, `authority_weighted_associative`; Theorems 26–28).

Axiom 3: composition_distributes_over_merge. States that $\otimes_s$ distributes over $\oplus_s$ (i.e., $(a \oplus_s b) \otimes_s c = (a \otimes_s c) \oplus_s (b \otimes_s c)$) for all strategies. **External validation:** Paper 3 Theorem 3; 10,000/100,000 pairs. This is the standard semiring distributivity law. For the min-plus tropical semiring it corresponds to the identity $\min(a, b) + c = \min(a + c, b + c)$, a classical result [?].

Axiom design pattern. Stating semiring laws as axioms follows the same pattern as Paper 22’s MSC mechanization, where `solver_functional` and `cael_causal_well_formed` capture properties of the production runtime that the abstract model cannot internally witness. In ATC, the axioms capture properties of the merge strategies that hold for all five strategies but whose proofs for `domainOverride` and `strictError` require case analysis on non-Nat carriers; stating them as axioms—with concrete Lean proofs for the three closed-form cases—is the appropriate mechanization discipline.

[TODO: If future work closes the remaining two strategy axioms (`domainOverride`, `strictError`) with closed-form Lean proofs, the axiom count drops from 3 to 3 kernel-only and this section can be updated accordingly.]

8 Related Work

Type-system composition. The foundational theory of type-system composition—most clearly expressed in the work of Reynolds [?] on parametricity and of Cardelli on subtyping lattices—establishes that well-typed composition preserves semantic properties. [TODO: cite: Reynolds 1983 “Types, Abstraction, and Parametric Polymorphism”, Cardelli–Wegner 1985] The composition theorem in this paper instantiates this principle for a specific two-component policy algebra.

Information-flow control. Sabelfeld and Myers [?] survey the information-flow control (IFC) literature, where the central challenge is composing security policies across multiple principals without leakage. [TODO: cite: Sabelfeld–Myers 2003 “Language-Based Information-Flow Security”] IFC composition typically relies both on non-interference, a property that our sandbox preservation theorem closely parallels: a denied capability is the analog of a secret that must not flow to a public output. The budget preservation theorem has no direct IFC counterpart—cost is a resource bound, not an information-flow property—making the two-component composition in ATC a distinct contribution.

Verified sandbox systems. Wahbe et al. [?] introduced software fault isolation (SFI) as an efficient sandboxing mechanism. Yee et al. [?] built Native Client, a verified SFI system for untrusted native code, with formal safety arguments. Haas et al. [?] extended these ideas to WebAssembly. Bosamiya et al. [?] demonstrated provably-safe WebAssembly sandboxing at USENIX Security 2022. These systems prove sandbox properties individually; ATC proves that combining a sandbox with a cost guard does not weaken either.

Mechanized security proofs in Lean/Coq. [TODO: cite: CompCert (Leroy 2009), CertiKOS (Gu et al. 2016), seL4 (Klein et al. 2009)] The practice of mechanizing security properties in proof assistants has produced significant results: CertiKOS [?] verified the composition of OS-level security policies; seL4 provided the first machine-checked proof of an OS kernel. ATC contributes a small but complete composition result in a newer proof assistant (Lean 4) aimed specifically at AI agent infrastructure.

Semiring frameworks for policy. Mohri [?] established the theoretical foundations of semiring-weighted automata, showing that the semiring laws (commutativity, associativity, distributivity) are the load-bearing algebraic properties for path-aggregation problems. Green, Karvounarakis, and Tannen [?] showed that semirings are the correct algebraic structure for database provenance—a direct ancestor of HoloScript’s provenance-merge strategies. ATC’s three semiring axioms are an instantiation of these foundations for the five W.GOLD.189 merge strategies.

MCP and tool-use security. Hou et al. [?] survey the MCP ecosystem and identify capability composition as an open security concern. Greshake et al. [?] and Zhan et al. [?] demonstrate injection and composition attacks on tool-integrated LLM agents. ATC addresses the composition concern for the specific case of sandbox+cost-guard composition, providing a mechanically verified guarantee.

9 Conclusion

We have presented ATC, a Lean 4 mechanization proving that the composition operator $\otimes$ preserves both the sandbox-restriction invariant and the budget-bounding invariant for AI agent tool execution. The mechanization comprises 23 theorems (0 sorry), 3 named axioms (all externally validated by Paper 3's CRDT semiring suite), and a direct production mapping to the HoloScript MCP server.

What is proved.

- No capability forbidden by the sandbox can be opened by composing with any cost guard (`sandbox_preservation`, `sandbox_preservation_for_all_cost_guards`).
- No over-budget cost guard can be cleared by composing with any sandbox policy (`budget_preservation`, `budget_preservation_for_all_sandbox_policies`).
- The composition operator forms a well-structured policy algebra: commutativity, identity, and annihilator laws all hold, enabling equational reasoning about policy chains.
- The four-valued execution result faithfully reflects both invariants in the observable outcome (`execute_preserves_sandbox`, `execute_preserves_budget`, `allowed_implies_both_permitted`).

Open: circuit-level enforcement layer. The composition theorem covers the logical structure of policy composition. Layer 2 (CAEL hash-chain audit of every composed execution) and Layer 3 (simulation-contract oracle) remain open for Lean mechanization and are targeted by Paper 4 [?] and the companion capstone [?].

Open: closing the remaining strategy axioms. The two strategies without closed-form Lean proofs (`domainOverride`, `strictError`) could be given concrete mechanizations with a richer carrier type; this would reduce the axiom count from three to zero. We leave this for future work.

Open: multi-component composition. The current theorem handles exactly two components (`sandbox` $\otimes$ `cost-guard`). An n -component composition theorem, admitting arbitrarily many policy layers stacked via $\otimes$, would generalize the result for multi-tenant fleet scenarios and is a natural extension.

Acknowledgments

This work is part of the HoloScript simulation-as-proof research program. The Lean mechanization builds on the provenance-semiring proofs in Paper 3 (CRDT). The composition operator design was informed by Paper 4 (Sandbox) and Paper 8 (Unified Transform).

Temporary page!

L^AT_EX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because L^AT_EX now knows how many pages to expect for this document.